

Prof. Dr. Stefan Leutenegger

Teaching Assistants: Tianyi Zhang, Cédric Le Gentil, and Simon Boche

Exercise 6: Nonlinear Least Squares (NLSQ) Optimization

Session date: 14/04/2026

Task 1: Polynomial Fitting and Gauss-Newton Optimization (Python)

Consider the M^{th} order polynomial in the parameter p :

$$z = \sum_{j=0}^M a_j p^j,$$

where you are supposed to estimate the coefficients a_j , i.e. $\mathbf{x} = [a_0, a_1, \dots, a_M]^T \in \mathbb{R}^{M+1}$. We are given a list of N measurements $\tilde{\mathbf{z}} = [\tilde{z}_1, \tilde{z}_2, \dots, \tilde{z}_N]^T$ and corresponding parameters $\tilde{\mathbf{p}} = [\tilde{p}_1, \tilde{p}_2, \dots, \tilde{p}_N]$.

i) With the sum of square errors:

$$c(\mathbf{x}) = \frac{1}{2} \sum_{i=1}^N \|\tilde{z}_i - z(\mathbf{x}, \tilde{p}_i)\|^2, \quad (1)$$

write down the solution for $\mathbf{x}$ that minimises it.

- ii) (Python) Implement the above scheme in `e06_least_squares.py`, where the measurements are already provided, and where the implementation should work for any $N > M$.
- iii) (Python) Unfortunately, we now have some outlier measurements. Change the outlier probability parameter `P_outlier` to a value > 0 , and observe the effect on your solution.
- iv) (**Advanced**, Python) To accommodate for outliers, we introduce the alternative of a robust cost function:

$$c(\mathbf{x}) = \sum_{i=1}^N \rho(\|\tilde{z}_i - z(\mathbf{x}, \tilde{p}_i)\|),$$

where $\rho(\cdot)$ stands for a robust cost function. Propose a new implementation to estimate $\mathbf{x}$ using Gauss-Newton. For initialisation you may still use the direct Least Squares Solution from above.

Background: The introduction of a robust cost function makes the error terms of (1) nonlinear, which is why we need to use iterative application of the Normal Equations, i.e. the Gauss-Newton scheme. In order to follow the recipe from Lecture 6, we introduce the error terms $e_i = \tilde{z}_i - h_i(\mathbf{x})$, where of course we are still dealing with a linear measurement function here, i.e. $h_i(\mathbf{x}) = \mathbf{H}_i \mathbf{x}$, with

$$\mathbf{H}_i = \begin{bmatrix} 1 & p_i & p_i^2 & \dots & p_i^M \end{bmatrix},$$

therefore we also get the Jacobian of the error term as $\mathbf{E}_i = -\mathbf{H}_i$.

Now for the robustification, we introduce different functions $\rho(r)$, with $r = \|\mathbf{e}_i\|$. As introduced in the lecture, we can now re-write the problem as

$$c(\mathbf{x}) = \sum_{i=1}^N \rho(\|\mathbf{e}_i\|) = \frac{1}{2} \sum_{i=1}^N \|\mathbf{e}'_i\|^2,$$

with $\mathbf{e}'_i = w(r)\mathbf{e}_i$, and $w(r) = \sqrt{2\rho(r)}/r$.

Typical choices for the robustifier are the Huber Loss ρ_H or the Cauchy Loss ρ_C :

$$\rho_H = \begin{cases} \frac{1}{2}r^2, & r \leq k, \\ k|r| - \frac{1}{2}k^2, & r > k, \end{cases} \quad \rho_C = \frac{c^2}{2} \log \left(1 + \frac{r^2}{c^2} \right),$$

with additional tunable parameters k and c . **Hint:** You will have to derive the new Jacobians $\mathbf{E}'_i$.

Task 2: 2D Motion Tracking System (Python)

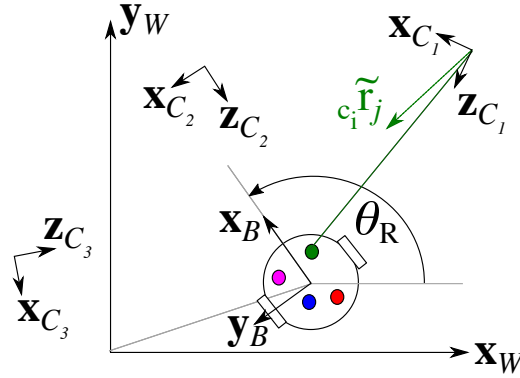


Figure 1: A two-wheel differential drive mobile robot with four fixed markers (indicated by the colorful dots on the robot base) observed by three fixed cameras measuring the bearing vectors $c_i \tilde{\mathbf{r}}_j$.

Consider the two-wheel differential drive mobile robot shown in Figure 1 with the following state:

$$\mathbf{x}^T = [x_R, y_R, \theta_R]^T,$$

where x_R, y_R denote the 2D position of the robot base frame in the world frame, and θ is the orientation of the robot frame relative to the world frame. The robot is equipped with 4 distinct markers (index j) at positions ${}_B\mathbf{t}_{M_j}$ (note the expression in the robot body coordinate frame $\mathcal{F}_B$). Now there are three fixed cameras (index i) located at poses $\mathbf{T}_{WC_i}$ that measure the bearing vectors (lengths normalised to 1) $c_i \tilde{\mathbf{r}}_j$ to those markers (2D in the x-y plane). Careful: the bearings are measured relative to the respective camera's individual coordinate frame.

- i) (Python) Open the Python script `e06_diff_drive_mocap.py` to see the set-up: the simulation will show the robot carrying out a 2D motion in a room.
- ii) Formulate the angular difference $e_{i,j}$ between the measured bearing ($c_i \tilde{\mathbf{r}}_j$) and the actual bearing to the marker j ($c_i \mathbf{t}_{M_j}$) which can be expressed as a function of the state and the known positions of the marker in the body frame, ${}_B\mathbf{t}_{M_j}$. Also write down the respective sum of squares cost function $c(\mathbf{x})$.
- iii) Derive the Jacobian $\mathbf{E}_{i,j} = \frac{\partial e_{i,j}}{\partial \mathbf{x}}$.
- iv) (Python) Using the Gauss-Newton optimisation scheme, implement an estimator for the robot state $\mathbf{x}$ that minimises the above cost $c(\mathbf{x})$ in `e06_diff_drive_mocap.py`.